Story Weaver

> AI 写作上下文流程图与泳道图见 docs/ai-context-flow.md。

Story Weaver 是一个面向长篇小说创作的 AI 辅助平台，当前项目已经包含可运行的前后端、局域网开发访问、AI 流式正文生成、世界观关联、RAG、模型服务管理，以及镜像发布与服务端部署链路。

当前能力

- 项目、章节、人物、剧情、因果、世界观、知识库的基础管理
- 章节正文 AI 拟生成、续写、扩写，支持流式返回
- Ollama 优先的模型服务管理与模型自动发现
- 项目与已有世界观关联
- 角色属性选择式组装与 AI 辅助生成
- 局域网开发访问
- Docker Hub / 阿里云 ACR / 自定义镜像仓库发布
- 服务端 `docker compose` 镜像部署

目录结构

```
story-weaver/
├─ backend/                    Spring Boot 后端
├─ front/                      Vue 3 + Vite + Vuetify 前端
├─ nginx/                      前端生产镜像 Nginx 配置
├─ sql/                        初始化与增量 SQL
├─ scripts/                    启动、发布脚本
├─ start-dev.bat
├─ start-dev.ps1
├─ start-dev.sh
├─ docker-compose.yml          本地容器化开发编排
├─ docker-compose.server.yml   服务端镜像部署编排
└─ .env.server.example         服务端环境变量示例
```

开发环境要求

- JDK 21+
- Maven 3.9+
- Node.js 20+
  说明：启动脚本会优先尝试 `.nvmrc` 中的版本；如果本机是 Node 22，也可以正常开发运行
- MySQL 8.x
- Redis 6.x+

当前开发环境默认连接：

- MySQL: `192.168.5.249:3306`
- Redis: `192.168.5.249:6379`

- 数据库用户: `root`
- 数据库密码: `Zhouwenjian:2871`
- Redis 密码: `Zhouwenjian:2871`

默认账号

- 璇蜂娇鎣－□鎒喟愚鍒嗛厤鐨勳处鍙凤紝鎴桄澽鈥滈处鍙风□鎒嗏€澶腑鍒涘缓 / 閎嶇疆璐冖彿

一键开发启动

Windows:

```
.\start-dev.bat
```

或者:

```
.\start-dev.ps1
```

Linux / macOS:

```
chmod +x start-dev.sh
./start-dev.sh
```

启动脚本会自动完成这些动作:

- 检查 `java`、`mvn`、`node`、`npm`
- 如果检测到 `nvm`，尝试切换到 `.nvmrc` 指定版本
- 检查 MySQL 和 Redis 连通性
- 如果 `8080` 或 `5173` 已被占用，先杀掉旧进程再重启
- 安装前端依赖
- 启动前后端并输出本机地址和局域网地址

默认开发访问地址:

- 前端: `http://localhost:5173`
- 后端: `http://localhost:8080/api`

说明:

- 前端默认走相对路径 `/api`
- 通过 `start-dev` 启动后，局域网中的其他设备也可以直接访问前端
- 当前后端已放开常见局域网网段的 CORS

手动开发启动

后端:

```
cd backend
mvn spring-boot:run
```

前端：

```
cd front
npm install
npm run dev
```

数据库初始化

全新库初始化：

```
mysql -u root -p < sql/001_init_database.sql
mysql -u root -p < sql/002_seed_data.sql
```

如果你使用的是较早版本的开发库，还需要按顺序补执行增量脚本：

```
mysql -u root -p < sql/003_align_legacy_dev_schema.sql
mysql -u root -p < sql/004_world_setting_and_naming_config.sql
mysql -u root -p < sql/005_world_setting_association_and_character_attributes.sql
mysql -u root -p < sql/006_account_security_and_user_management.sql
mysql -u root -p < sql/007_character_reuse_and_chapter_binding.sql
mysql -u root -p < sql/008_outline_module.sql
```

Docker 相关文件说明

- `docker-compose.yml`
  用于本地容器化开发，不建议直接作为生产部署文件使用
- `docker-compose.server.yml`
  用于服务端通过镜像部署，内置 `gateway` Nginx 做 HTTPS 终止
- `scripts/deploy.bat`
- `scripts/deploy.ps1`
- `scripts/deploy.sh`
  用于将前后端镜像发布到所选镜像仓库
- `.deploy/registry.env.example`
  发布脚本的配置示例文件

当前仓库已经按阿里云 ACR 预设好这些默认值：

- `REGISTRY_PROVIDER=aliyun-acr`
- `REGISTRY_HOST=crpi-2iicgf8z27uyvaq1.cn-hangzhou.personal.cr.aliyuncs.com`
- `REGISTRY_NAMESPACE=silvericekey`
- `LOGIN_USERNAME=z516798599@qq.com`
- `BACKEND_IMAGE_NAME=story-weaver-backend`

- `FRONTEND_IMAGE_NAME=story-weaver-front`

也就是说，首次发布时你通常只需要确认镜像标签，然后输入 `docker login` 密码即可。

发布与部署概览

1. 本地运行发布脚本
2. 首次执行时交互式选择镜像仓库类型
3. 构建并推送前后端镜像
4. 在服务端准备 `.env`
5. 使用 `docker-compose.server.yml` 拉取镜像并启动

当前服务端部署默认值：

- 域名：`home.silvericekey.fun`
- 证书目录：`/usr/local/project/cer`
- 证书文件：`fullchain.pem` + `privkey.pem`
- 外部端口：`${HTTP_PORT:-80}` / `${HTTPS_PORT:-443}`

当前发布脚本支持三类仓库：

- Docker Hub
- 阿里云 ACR
- 自定义 Docker Registry

发布完成后，脚本还会直接输出一份 Dockge 可用的 `compose.yaml` 模板。

说明：

- `volumes` 中所有以 `xxx` 开头的路径，表示需要替换成服务器本地实际目录
- 输出内容可以直接复制到 Dockge 的 `compose.yaml`

如果你直接用当前这套阿里云仓库，最简单的用法是：

```
.\scripts\deploy.bat
```

或：

```
./scripts/deploy.sh
```

镜像标签现在按版本号管理：

- 默认读取根目录 `VERSION`，当前为 `1.0.0`
- 发布脚本会拒绝 `latest`、`test` 这类漂移标签
- 建议发布时显式使用版本号，例如 `1.0.1`、`1.0.1-beta.1`

示例：

```
.\scripts\deploy.bat -Tag 1.0.1
```

```
./scripts/deploy.sh --tag=1.0.1
```

详细步骤见 DEPLOYMENT.md。

常见说明

1. 为什么局域网访问前端时之前会报跨域

旧版本前端把 API 地址写死成了 `http://localhost:8080/api`。
对于局域网其他设备来说，这会变成“访问它自己本机的 8080”，自然会失败。现在默认改成了相对 `/api`，并由前端开发代理或生产 Nginx 统一转发。

2. 为什么正文生成现在不会一直转圈

流式正文生成现在是：

- 后端识别模型流结束信号
- 前端在收到 `complete` 事件后立刻结束等待
- 生产 Nginx 已关闭流式缓冲

所以不会再出现“最后已经有内容了，但页面还一直加载”的问题。

3. 开发环境与生产环境的前端 API 路径

- 开发环境：Vite 代理 `/api -> http://localhost:8080`
- 生产环境：Nginx 代理 `/api -> backend:8080`

这也是为什么现在推荐统一使用相对 `/api`。